

```
#!/usr/bin/env bash
# restore_db_rds.sh - RDS point-in-time
restore into a NEW, private instance.
# Never modifies the source instance,
Terraform state, or running containers.
#
#   restore_db_rds.sh --target-time 2026-
10-04T12:00:00Z [options]
#   restore_db_rds.sh --latest [options]
#
# Options:
#   --source ID      source instance
#                     (default: $SOURCE_DB or security-audit-
#                     prod-db)
#   --target ID      new instance id
#                     (default: <source>-pitr-<UTC stamp>)
#   --region R       AWS region (default:
#                     $AWS_REGION / CLI config)
#   --dry-run        read-only preflight,
#                     print the restore command, change nothing
#   --yes            skip the confirmation
#                     prompt
#
# In-region only. Cross-region recovery:
# see runbook section 6.5.
set -euo pipefail

SOURCE="${SOURCE_DB:-security-audit-prod-
db}"
TARGET=""
```

```

TARGET_TIME=""
USE_LATEST=false
DRY_RUN=false
ASSUME_YES=false
REGION="${AWS_REGION:-${AWS_DEFAULT_REGION:-}}}"

die() { echo "ERROR: $" >&2; exit 1; }

while [[ $# -gt 0 ]]; do
    case "$1" in
        --target-time) TARGET_TIME="${2:-}";
    shift 2 ;;
        --latest)      USE_LATEST=true; shift
    ;;
        --source)      SOURCE="${2:-}"; shift 2
    ;;
        --target)      TARGET="${2:-}"; shift 2
    ;;
        --region)      REGION="${2:-}"; shift 2
    ;;
        --dry-run)     DRY_RUN=true; shift ;;
        --yes)         ASSUME_YES=true; shift
    ;;
        -h|--help)     sed -n '2,16p' "$0";
    exit 0 ;;
        *)             die "unknown option: $1"
    ;;
    esac
done

if [[ "$USE_LATEST" == true && -n
"$TARGET_TIME" ]]; then die "use --latest
OR --target-time, not both"; fi

```

```

if [[ "$USE_LATEST" == false && -z
"$TARGET_TIME" ]]; then die "--target-time
(ISO 8601 with Z/offset) or --latest is
required"; fi
if [[ -n "$TARGET_TIME" ]] && ! [[
"$TARGET_TIME" =~ ^[0-9]{4}-[0-9]{2}-[0-9]
{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}(Z|[-+][0-9]
{2}:[0-9]{2})$ ]]; then
    die "bad --target-time '$TARGET_TIME'
(want e.g. 2026-10-04T12:00:00Z)"
fi
command -v aws >/dev/null || die "aws CLI
not found"
command -v python3 >/dev/null || die
"python3 not found"

# computed ONCE so every step refers to the
same instance id
TARGET="${TARGET:-${SOURCE}-pit-$(date -u
+%Y%m%d%H%M%S)}"
AWS=(aws)
[[ -n "$REGION" ]] && AWS+=(--region
"$REGION")

epoch() {
    python3 -c 'import sys; from datetime
import datetime;
print(int(datetime.fromisoformat(sys.argv[1]
.replace("Z", "+00:00")).timestamp()))'
"$1"
}

# ---- read-only preflight -----
-----

```

```

echo "Preflight: ${SOURCE}"
INFO="$(("${AWS[@]}") rds describe-db-
instances --db-instance-identifier
"$SOURCE" --output json)" \
    || die "cannot describe '$SOURCE' (wrong
id, region, or permissions)"

mapfile -t F < <(python3 -c '
import json, sys
d = json.load(sys.stdin)["DBInstances"][0]
print(d["DBInstanceStatus"])
print(d.get("LatestRestorableTime", ""))
print(d.get("BackupRetentionPeriod", 0))
print(d["DBSubnetGroup"]
["DBSubnetGroupName"])
print(" ".join(g["VpcSecurityGroupId"] for
g in d["VpcSecurityGroups"]))
' <<<"$INFO")

STATUS="${F[0]}"; LATEST="${F[1]}";
RETENTION="${F[2]}"; SUBNET_GROUP="${F[3]}"
read -r -a SG_ARR <<<"${F[4]}"

[[ "$STATUS" == "available" ]] || die
"source status is '$STATUS', expected
'available'"
(( RETENTION >= 1 )) || die
"BackupRetentionPeriod=0 on source: PITR is
not possible"
[[ -n "$LATEST" ]] || die "source reports
no LatestRestorableTime"
(( ${#SG_ARR[@]} > 0 )) || die "source has
no VPC security groups"

```

```

if [[ "$USE_LATEST" == false ]]; then
    T="$(epoch "$TARGET_TIME")"; L="$(epoch
"$LATEST")"; NOW="$(date -u +%s)"
    (( T <= NOW )) || die "target time is in
the future"
    (( T <= L )) || die "target time is
after LatestRestorableTime ($LATEST)"
    (( T >= NOW - RETENTION * 86400 )) || die
"target time is older than the
${RETENTION}-day retention window"
fi
echo "  status=$STATUS
retention=${RETENTION}d
latest_restorable=$LATEST"
echo "  new instance will be private, in
subnet group '$SUBNET_GROUP', SGs:
${SG_ARR[*]}"

cmd=("${AWS[@]}") rds restore-db-instance-
to-point-in-time
    --source-db-instance-identifier "$SOURCE"
    --target-db-instance-identifier "$TARGET"
    --db-subnet-group-name "$SUBNET_GROUP"
    --vpc-security-group-ids "${SG_ARR[@]}"
    --no-publicly-accessible
    --tags "Key=restored-from,Value=$SOURCE"
    "Key=restore-target,Value=${TARGET_TIME:-
latest}")
if [[ "$USE_LATEST" == true ]]; then cmd+=
(--use-latest-restorable-time); else cmd+=
(--restore-time "$TARGET_TIME"); fi

if [[ "$DRY_RUN" == true ]]; then
    echo "[DRY RUN] would run:"

```

```

    printf '  %q' "${cmd[@]}"; echo
    exit 0
fi

if [[ "$ASSUME_YES" == false ]]; then
    read -r -p "Create '$TARGET' from
'$SOURCE' at ${TARGET_TIME:-latest}?
(yes/no): " ok
    [[ "$ok" == "yes" ]] || { echo
"Cancelled."; exit 0; }
fi

# ---- restore -----
-----
"${cmd[@]}" >/dev/null
echo "Restore started: $TARGET (this takes
a while)"
"${AWS[@]}" rds wait db-instance-available
--db-instance-identifier "$TARGET"

ENDPOINT="$(("${AWS[@]}" rds describe-db-
instances --db-instance-identifier
"$TARGET" \
    --query 'DBInstances[0].Endpoint.Address'
--output text)"

cat <<EOF

Restored: $TARGET
Endpoint: $ENDPOINT

Next (manual, deliberately not automated):
  1. Validate row counts on the new
endpoint (psql -h $ENDPOINT ...).

```

```
2. Update the DB URL in your secret
manager; restart dashboard + scheduler.
3. Keep '$SOURCE' until the incident is
closed.
4. Reconcile Terraform afterwards
(import/rename). Never from a restore
script.
EOF
```